

## Lab 3: A Short Fake-News Pipeline

Clean two news CSVs and walk them through the pipeline.

Work in a **Jupyter** notebook. Number the exercises. Do notes **2.3**, **3.1**, **3.2**, and **3.3** first. You will need `pandas`, `nltk`, and `scikit-learn` (or another library you already know).

Download [fakeNews.csv](#) and [trueNews.csv](#) into the same folder as the notebook (or use a relative path).

When you are done: **File** → **Download as** → **HTML**, then upload the HTML on Canvas. Save the notebook before you export.

These files are COVID-era claims: fact-checked **false / misleading** posts and **true** news posts. The job is binary classification, not sentiment.

```
import pandas as pd
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

nltk.download("punkt")
nltk.download("wordnet")
```

---

### 1. Import (1 point)

1. Read `fakeNews.csv` and `trueNews.csv` with `pandas`.
2. Print column names, `shape`, and two sample rows from each file.
3. Build **one** working frame with at least: - a text column (use `Text`) - a binary label (`0` = fake / misleading, `1` = true) - `Country` if you will slice by country in Exercise 5

`fakeNews.csv` has `Binary Label`. `trueNews.csv` has `Label`. Align the names before you concatenate. Drop rows with empty text.

---

### 2. Clean and lemmatize (1 point)

On the text column:

1. Remove the `#` character from hashtags (`#Covid` → `Covid`). Keep the token.
2. Tokenize. Drop tokens of length **less than 2**. Drop tokens that are only punctuation.
3. Lemmatize with `WordNetLemmatizer`. Join the lemmas back into a string (or keep a list — stay consistent).

Print **one** original text and its cleaned version.

Do not lowercase away information you still want to inspect; if you lowercase, do it on purpose and say so.

---

### 3. Train (1 point)

1. Split **70% train / 30% test**. Stratify on the label if you can (`train_test_split(..., stratify=y)`).
2. Train a classifier. Bag-of-words or TF-IDF plus logistic regression (or Naive Bayes) is enough. Embeddings are allowed. You may use a library.
3. Print the vectorizer and model you chose, and the train/test sizes.

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
```

Fit **only** on the training split. Transform the test set with the same vectorizer.

---

### 4. Intrinsic metrics (1 point)

On the test set, compute and print:

- accuracy
- precision
- recall
- F1

Keep the file labels (`0` = fake / misleading, `1` = true) unless you recode — if you recode, say so. Use `sklearn.metrics` and show a classification report. For `precision_score` / `recall_score` / `f1_score` on two classes, set `average` (e.g. `macro` or `binary` with an explicit `pos_label`).

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report, confusion_matrix
```

---

### 5. Extrinsic-style check (1 point)

Pick **one** and actually do it (not a one-line “I could...”):

- print a confusion matrix and read three false positives and three false negatives

- or compare F1 (or error rate) across `Country` / `Region` on the test rows
- or list the tokens with the largest coefficients if you used a linear model

Write two or three sentences: what the model is actually using, and one failure that a metric average hid.